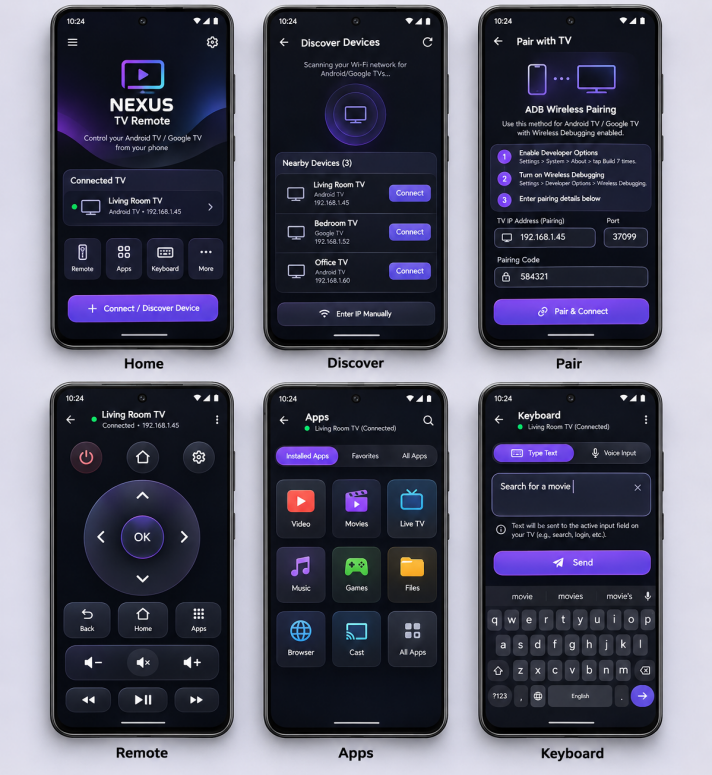
NEXUS

TV REMOTE

Android & Google TV control over local Wi-Fi
Native Android application • Kotlin • Jetpack Compose

PROJECT BLUEPRINT & IMPLEMENTATION SPECIFICATION

From first Android Studio project to a tested, installable application



Generated concept UI — Home, discovery, pairing, remote, apps and keyboard. These are visual design references, not screenshots of a functioning application.

Project direction

Build a phone-native Android application that controls compatible Android TV and Google TV devices on the same local network. The first release prioritizes secure pairing, dependable navigation, media controls and app launching. Advanced features are staged behind technical validation.

Document version 1.0 • Prepared 8 November 2026 • Status: Proposed implementation plan

1. Executive summary

NEXUS TV Remote is a native Android application intended to turn an Android phone into a high-control remote for compatible Android TV and Google TV devices. The phone and television communicate over the same local Wi-Fi network. The initial control channel is Android Debug Bridge (ADB) over Wi-Fi, using Android's modern Wireless Debugging pairing flow where supported.

Unlike a conventional infrared remote, an authenticated ADB host can request Android shell services such as key events, package/activity operations and display capture. However, ADB is a developer interface—not a consumer remote-control API—and availability depends on TV firmware, user settings, OEM restrictions and Android version.

> **Critical feasibility point**
> An Android app cannot simply call `adb shell` as though it were running on a desktop. NEXUS must include or integrate an ADB host/client implementation capable of pairing, storing host identity securely, negotiating the transport and opening shell services. Validate this on a real TV before building the complete product UI.

The recommended delivery strategy is a staged native application: first prove direct phone-to-TV ADB connectivity; then build a stable command layer and remote UI; then add app discovery, text entry, screenshots and multi-device management. If direct on-device ADB proves unreliable on a target TV, the architecture can optionally support a local companion gateway as a fallback, but the primary product remains a Kotlin Android app.

1.1 Project goals

- Discover compatible Android TV / Google TV devices on the user's local network, with manual IP/port entry as a fallback.
- Pair with a TV using its Wireless Debugging pairing code and pairing endpoint; establish a separate authenticated ADB connection endpoint.
- Provide a responsive remote interface: D-pad, OK, Back, Home, volume, mute and playback controls.
- Launch supported installed TV applications and provide keyboard text entry where the target input method accepts ADB text injection.
- Support optional screenshot capture, app listing and saved device profiles after the core connection is stable.
- Keep the first release local-first: no account, cloud backend, paid API or mandatory external server.

1.2 Non-goals for the first release

- Controlling Samsung Tizen, LG webOS or Roku through ADB. Those platforms do not expose Android's ADB control surface.
- Root access, modifying protected system partitions, bypassing DRM or defeating app security restrictions.
- Guaranteeing TV power-on from deep standby. ADB may be unavailable when the TV is asleep or its network interface is powered down.
- Guaranteeing screen capture of protected video. DRM-protected content may appear black or be omitted.
- Promising compatibility with every Android TV model or firmware.

2. Product definition and scope

2.1 Target users and usage scenario

A user installs NEXUS on an Android phone, connects the phone and television to the same Wi-Fi network, enables Wireless Debugging on the TV, pairs once, and then uses the phone as a remote. The app stores the TV profile and the ADB host key material securely so that a later connection can be re-established when the TV advertises or exposes its ADB connection service.

2.2 Target platform

Area	Initial target / decision
Phone OS	Android; choose a conservative minSdk after validating the ADB library and Compose dependencies. A practical starting proposal is API 26+ (Android 8.0), subject to library compatibility.
TV OS	Android TV and Google TV devices with user-accessible Developer Options and Wireless Debugging or legacy ADB-over-TCP.
Network	Same trusted Wi-Fi LAN; no internet required for basic remote control.
Development IDE	Android Studio stable channel; use the bundled Android SDK, emulator, Gradle and Kotlin tooling.
UI toolkit	Jetpack Compose + Material 3; single-activity architecture.
Persistence	DataStore for settings; Room only if richer device/history data is required; Android Keystore-backed encryption for sensitive host credentials.
Build output	Debug APK for internal testing; signed release APK/AAB only after compatibility and security review.

2.3 Feature priorities

Priority	Feature	Acceptance definition
P0	Manual connection + ADB pairing	User can enter TV pairing IP/port and 6-digit code; pairing succeeds and returns a clear result.
P0	Remote navigation	Up/down/left/right/OK/back/home commands work on a validated TV.
P0	Volume + media keys	Supported volume and media actions work; unsupported actions are reported cleanly.
P1	Device profiles	User can save a friendly name and reconnect to a previously paired TV.
P1	App launcher	Installed package list can be retrieved and selected apps can be launched where permitted.
P1	Text entry	User can send supported text to the focused TV input field; unsupported characters are handled.
P2	Screenshot viewer	Capture and display TV screenshots on supported devices; clearly disclose DRM limitations.
P2	Network discovery	Discover ADB service advertisements where visible; manual connection remains available.
P3	Touchpad / gestures	Map swipes/taps to navigation only after latency and repeat behaviour are tuned.
P3	Multiple TVs	Manage multiple paired devices and switch active target.

3. System architecture

The product should be organized around a transport-independent TV control interface. UI components should not know ADB packet details, socket endpoints or keycode values. A domain command is translated by the ADB adapter into a supported remote shell action.

Jetpack Compose UI
Home • Discover • Pair • Remote • Apps • Keyboard • Settings

Presentation layer
ViewModels • UI state • events • navigation

Domain layer
TVCommand • TVDevice • PairingUseCase • SendCommandUseCase • DeviceRepository

Data / infrastructure layer
ADB client adapter • discovery • secure credential store • DataStore / Room

Local network transport
TCP + TLS for modern Wireless Debugging; legacy ADB TCP only when explicitly enabled and supported

Android TV / Google TV
adbd • shell service • input manager • package manager • screencap

3.1 Connection lifecycle

1. User selects Add TV and chooses Wireless Debugging pairing.
2. App explains that the phone and TV must be on the same trusted Wi-Fi network and asks the user to enable Wireless Debugging on the TV.
3. User enters the TV's pairing IP address, pairing port and six-digit pairing code. The pairing port is temporary and may change.
4. ADB client performs the Wireless Debugging pairing protocol and securely persists the host identity / keys.
5. App obtains or discovers the separate ADB TLS connection endpoint. Do not assume the pairing port and connection port are the same.
6. App establishes an authenticated ADB transport and opens only the required shell services.
7. User sends remote actions; the command layer maps the selected TV action to an allowlisted ADB shell operation.
8. On disconnect, the app reports state, retries with bounded backoff and lets the user re-pair if the TV revoked trust.

3.2 Modern Wireless Debugging vs legacy TCP/IP

Mode	How it works	Recommendation
Wireless Debugging (Android 11+)	User initiates pairing on TV; pairing code + temporary pairing port; then separate TLS connection endpoint and authenticated host key.	Primary path. Encrypted and designed for wireless debugging.
Legacy ADB TCP/IP	ADB daemon listens on a TCP port, commonly 5555 after explicit enablement or vendor configuration; no modern pairing-code workflow.	Optional compatibility path only. Avoid instructing users to leave unauthenticated/debug ports exposed.
Desktop ADB bridge	A computer/Raspberry Pi runs official platform-tools and acts as ADB host; phone app sends commands to a local authenticated bridge API.	Fallback if direct on-phone ADB client is not production-ready.

Do not hardcode port 5555
Modern Wireless Debugging uses dynamically assigned ports. The TV presents a pairing endpoint and, after pairing, a distinct connect endpoint. Discovery may use mDNS service types such as `_adb-tls-pairing._tcp` and `_adb-tls-connect._tcp`; availability varies by network and device.

4. Free technology stack

All recommended core tools are available without a paid subscription. Some open-source libraries have their own licenses and maintenance status; review each repository's current license, release history and Android compatibility before shipping.

Layer	Recommended technology	Purpose / cost
IDE & SDK	Android Studio + Android SDK + Platform Tools	Official Android development environment; free.
Language	Kotlin	Primary app language; free and open source.
UI	Jetpack Compose + Material 3	Declarative native UI; Google-maintained Android libraries.
Architecture	MVVM + repository pattern; optional Clean Architecture boundaries	Testable separation of UI, domain and device transport.
Async	Kotlin Coroutines + Flow	Non-blocking network work and observable connection state.
Dependency injection	Manual DI initially; Hilt optional	Avoid adding complexity before core connection works.
ADB implementation	Evaluate `adb-kt` (third-party Kotlin ADB client) or another maintained ADB host library	Candidate for direct on-phone ADB; technical spike required before commitment.
Networking	Java/Kotlin sockets; Okio optional	ADB transport; do not substitute plain HTTP for ADB protocol.
Discovery	Android NSD / mDNS APIs	Discover advertised local services; manual IP/port entry fallback.
Persistence	DataStore; Room optional	Preferences and saved device profiles.
Secrets	Android Keystore + encrypted storage wrapper	Protect ADB host private key material.
Testing	JUnit, kotlinx-coroutines-test, MockK or Mockito, Compose UI Test	Unit, integration and UI tests; free.
Version control	Git + GitHub free tier or local Git	Source history and collaboration; optional hosted account.
Design assets	Figma Starter / Penpot / Android Studio Preview	UI design and review; free tiers / open source.

4.1 ADB library decision: mandatory proof-of-concept

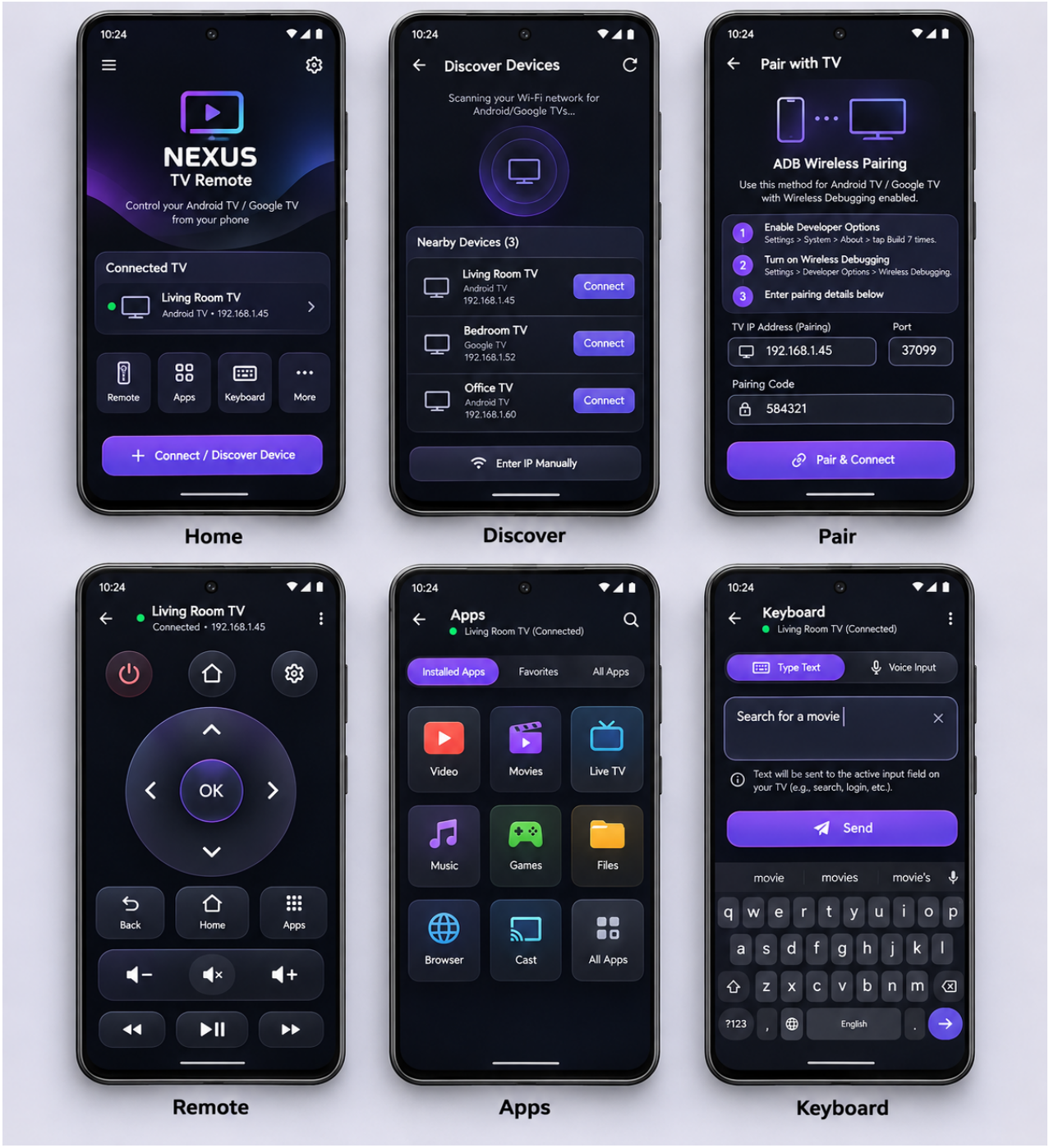
A pure Kotlin ADB client project such as `adb-kt` advertises Android support and modern Wireless Debugging pairing, but it is a third-party dependency—not an official Android SDK API. The official Android Studio `adblib` is designed as a library for ADB host environments and is not automatically a drop-in Android application dependency. NEXUS should compare candidates in a short prototype before selecting one.

Evaluation criterion	What to verify
Android runtime support	Can it run inside an ordinary Android app process, on the chosen minSdk and target SDK?
Modern pairing	Does it implement the Wireless Debugging pairing flow, TLS transport and key persistence?
Command support	Can it open shell services, stream stdout/stderr and close sessions cleanly?
Security	How are private keys generated, stored, rotated and deleted? Is TLS certificate validation correct?
Maintenance	Recent commits/releases, issue activity, documented API and reproducible build.
License	License is compatible with the intended distribution; include notices and attribution.
TV compatibility	Test on at least two different Android/Google TV vendors and firmware versions.

Cost model
No cloud backend, subscription API, paid database, or hosted service is required for the core local-network remote. Costs may still arise from optional hardware, test devices, internet access, or paid app-store developer registration if the app is later published.

5. UI / UX specification

The interface direction is a premium dark remote: deep navy/black surfaces, restrained purple accent, high-contrast controls, large touch targets and clear connection status. The design should remain usable one-handed and should not require the user to understand ADB terminology after pairing.



Generated visual concept: six proposed screens. Treat the mockup as a design reference; final Compose layouts should use responsive constraints and real Android typography, not reproduce poster labels or sample IP addresses literally.

5.1 Screen inventory

Screen	Main content	Key interactions
Welcome / Home	NEXUS branding, active TV card, connection state, quick actions	Connect, discover, open remote, settings
Discover Devices	Discovered TV list, scanning state, manual IP entry	Select device, retry scan, enter address

Screen	Main content	Key interactions
Pair with TV	Setup instructions, pairing IP, pairing port, 6-digit code	Validate fields, start pairing, show success/error
Remote	Power attempt, Home, Back, D-pad, OK, volume, mute, playback	Tap, long press/repeat, haptic feedback optional
Apps	Installed apps list, favorites, search	Launch selected package/activity
Keyboard	Text field, send action, clear, supported-character note	Send text to active TV input
Screenshot	Latest capture preview, refresh/save/share	Capture, display, save locally
Settings	Saved TVs, reconnect, rename, remove pairing, privacy info	Manage device profiles and permissions

5.2 Interaction rules

- Show a persistent connection indicator: Connected, Connecting, Disconnected, Pairing required, or Unsupported.
- Disable controls while the device is disconnected; never silently drop a command.
- Use large D-pad and volume hit areas with accessible content descriptions.
- Support press-and-hold repeat for D-pad and volume only after command throttling is implemented.
- Provide visible feedback for failed commands and preserve the TV selection when navigating between screens.
- Use confirmation before removing a saved TV or deleting its trusted host credentials.
- Never display a real pairing code or host private key in logs, analytics or crash reports.

6. User journey and TV setup

6.1 First-time pairing flow

1. Install NEXUS on the phone and open it.
2. Connect phone and TV to the same Wi-Fi network. Avoid guest networks or client isolation.
3. On the TV, open Settings and locate About / System information. Tap Build or Build number repeatedly (commonly seven times) to enable Developer Options; menu wording differs by manufacturer.
4. Open Developer Options and enable Wireless Debugging. The TV may show an authorization prompt; the user must explicitly allow it.
5. Open Wireless Debugging and choose Pair device with pairing code. The TV displays an IP address, a temporary pairing port and a six-digit code.
6. In NEXUS, enter those values and start pairing. The app must distinguish the pairing endpoint from the later connection endpoint.
7. After successful pairing, establish the ADB connection using the current connect endpoint, or discover it via the advertised TLS connect service.
8. Send a safe test command (for example, Home) and show the user a successful connection state.

6.2 TV settings variability

Android TV and Google TV menu labels vary across OEMs and OS versions. Sony, TCL, Xiaomi, Hisense, Philips and other vendors may move Developer Options under different Settings categories. The app should include adaptable guidance and a manual 'I cannot find this setting' help path rather than hardcoding a single menu path.

> **Security guidance for end users**
> Wireless Debugging gives the paired host developer-level control. Users should pair only on a trusted home network, never share pairing codes, disable Wireless Debugging when not needed, and revoke debugging authorizations if the phone is lost or the app is no longer trusted.

6.3 Disconnect and recovery cases

Condition	App response
TV changes Wi-Fi / IP	Re-discover services; allow manual re-entry; preserve profile and host key.
TV reboots	Attempt reconnect after service advertisement returns; bounded retry only.
Wireless Debugging turned off	Show 'Wireless Debugging is off' and guide user to re-enable it.
Host authorization revoked	Show re-pair action; do not loop on authentication failure.
Pairing code expires	Ask user to generate a fresh code and port on the TV.
Different Wi-Fi / guest network	Explain same-LAN requirement and possible AP/client isolation.
ADB library error	Show a user-safe error code; store diagnostic details without secrets.

7. Command model and ADB operations

Use a domain-level command enum and a single ADB adapter. UI code should call `sendCommand(TvCommand.HOME)` rather than constructing shell strings. The adapter must map known actions to validated arguments and reject arbitrary user-provided shell commands.

Action	Android key event	Typical ADB shell form
Home	KEYCODE_HOME (3)	input keyevent 3
Back	KEYCODE_BACK (4)	input keyevent 4
D-pad Up	KEYCODE_DPAD_UP (19)	input keyevent 19
D-pad Down	KEYCODE_DPAD_DOWN (20)	input keyevent 20
D-pad Left	KEYCODE_DPAD_LEFT (21)	input keyevent 21
D-pad Right	KEYCODE_DPAD_RIGHT (22)	input keyevent 22
OK / Select	KEYCODE_DPAD_CENTER (23)	input keyevent 23
Volume Up	KEYCODE_VOLUME_UP (24)	input keyevent 24
Volume Down	KEYCODE_VOLUME_DOWN (25)	input keyevent 25
Power	KEYCODE_POWER (26)	input keyevent 26 (may be blocked or behave differently)
Mute	KEYCODE_VOLUME_MUTE (164)	input keyevent 164 (device-dependent)
Play/Pause	KEYCODE_MEDIA_PLAY_PAUSE (85)	input keyevent 85
Rewind / Fast forward	KEYCODE_MEDIA_REWIND (89) / FAST_FORWARD (90)	input keyevent 89 / 90

7.1 Example Kotlin domain model

```
enum class TvCommand {
    HOME, BACK, UP, DOWN, LEFT, RIGHT, SELECT,
    VOLUME_UP, VOLUME_DOWN, MUTE, POWER,
    PLAY_PAUSE, REWIND, FAST_FORWARD
}

interface TvControlClient {
    suspend fun connect(device: TvEndpoint): Result<Unit>
    suspend fun pair(request: PairingRequest): Result<Unit>
    suspend fun send(command: TvCommand): Result<Unit>
    suspend fun sendText(text: String): Result<Unit>
    suspend fun captureScreenshot(): Result<ByteArray>
    suspend fun disconnect()
}
```

7.2 Example mapping (illustrative)

```
private val keyEvents = mapOf(
    TvCommand.HOME to 3,
    TvCommand.BACK to 4,
    TvCommand.UP to 19,
    TvCommand.DOWN to 20,
    TvCommand.LEFT to 21,
    TvCommand.RIGHT to 22,
    TvCommand.SELECT to 23,
    TvCommand.VOLUME_UP to 24,
    TvCommand.VOLUME_DOWN to 25,
    TvCommand.POWER to 26,
    TvCommand.MUTE to 164,
    TvCommand.PLAY_PAUSE to 85
)

// ADB adapter only: never expose arbitrary shell strings to the UI.
suspend fun send(command: TvCommand): Result<Unit> {
    val keyCode = keyEvents[command]
        ?: return Result.failure(UnsupportedOperationException())
    return adbShell("input keyevent $keyCode")
```

7.3 Text input and app launching

- Text injection through `input text` is not equivalent to a full keyboard API. Spaces, punctuation, Unicode, shell escaping and the TV's current focused field need explicit testing.
- Never interpolate raw text into a shell command. Use a tested escaping routine or a library API that safely encodes arguments; define a clear supported character set for the MVP.
- Do not assume package/activity names are universal. Retrieve installed package information where possible, allow user favourites, and validate launch targets.
- An app can be launched with Android package manager / activity manager shell operations when permitted. Prefer package launch intent resolution over hardcoded activity class names.

7.4 Screenshot flow

ADB supports `screencap` on the target device and can stream or retrieve the resulting PNG. Implement it as an optional feature: capture to a temporary location or stream output, transfer bytes through the ADB client, decode with Android Bitmap APIs, then delete temporary files. Protected video content may be blacked out by platform security.

8. Security, privacy and Android permissions

NEXUS is a local-network control tool. ADB host credentials are sensitive because a trusted host can perform privileged debugging actions on the TV. Security is a core product requirement, not a later enhancement.

8.1 Security requirements

- Pair only after an explicit user action and a clear explanation of the control granted.
- Use modern Wireless Debugging TLS pairing and encrypted ADB transport when supported.
- Generate and persist host identity material using a vetted ADB implementation; protect private key material with Android Keystore-backed encryption where compatible.
- Do not log pairing codes, private keys, raw shell command payloads or sensitive TV data.
- Expose only fixed, allowlisted commands. Do not implement a general-purpose shell console in the consumer UI.
- Use connection timeouts, bounded retries, command throttling and cancellation.
- Provide Remove device / Forget pairing functionality and clear local credentials when requested.
- Keep all network operations on Dispatchers.IO or an appropriate coroutine dispatcher; never block the main thread.

8.2 Permissions and platform evolution

The app will need INTERNET for network sockets. Nearby Wi-Fi and local-network permission rules evolve across Android releases. As of the current Android developer guidance, Android 17 (API 37) introduces mandatory local-network protection for apps targeting API 37+, with ACCESS_LOCAL_NETWORK required for broad local-network communication. Earlier releases have different behaviour and transitional guidance. Re-check the official permission page when choosing targetSdk and preparing each release.

Permission / API	Why it may be needed	Implementation note
INTERNET	TCP sockets to the TV	Normal manifest permission; no runtime dialog.
ACCESS_NETWORK_STATE	Read current network connectivity	Useful for showing Wi-Fi/network status.
NEARBY_WIFI_DEVICES	Some Wi-Fi-related operations on newer Android versions	Only request if the APIs used require it; do not request broad permissions without rationale.
ACCESS_LOCAL_NETWORK	Android 17 / API 37+ broad LAN communication	Declare and request at runtime when targeting API 37+ and making direct LAN connections, subject to current guidance.
NSD / mDNS	Discover advertised ADB services	Use Android NSD APIs; discovery may be affected by permission and network policies.
Multicast handling	Some mDNS implementations	Only acquire multicast lock if actually required; release it promptly.

Permission policy
Do not copy a permission manifest from an old tutorial and assume it is future-proof. Confirm the current Android SDK behaviour for the selected targetSdk. Prefer system-mediated discovery pickers where they meet the use case; otherwise explain and request the local-network permission when required.

8.3 Privacy statement for the app

Proposed privacy posture: no account, no cloud telemetry by default, no remote server, no advertising SDK and no collection of TV usage history. Saved TV names, addresses and connection metadata remain on the phone. Users can remove profiles and credentials from Settings. If analytics or crash reporting is introduced later, make it opt-in and never include ADB secrets.

9. Android Studio project structure

Start with a single Gradle app module. Keep the initial architecture modular by package rather than splitting into multiple Gradle modules prematurely. Extract modules only when the ADB adapter or domain layer becomes independently reusable.

```
NexusTvRemote/
├── app/
│   ├── src/main/
│   │   ├── java/com/example/nexus/
│   │   │   ├── MainActivity.kt
│   │   │   ├── NexusApp.kt
│   │   │   ├── navigation/
│   │   │   │   ├── AppNavHost.kt
│   │   │   │   └── Routes.kt
│   │   │   ├── ui/
│   │   │   │   ├── theme/
│   │   │   │   ├── components/
│   │   │   │   ├── home/
│   │   │   │   ├── discovery/
│   │   │   │   ├── pairing/
│   │   │   │   ├── remote/
│   │   │   │   ├── apps/
│   │   │   │   ├── keyboard/
│   │   │   │   └── settings/
│   │   │   ├── domain/
│   │   │   │   ├── model/
│   │   │   │   ├── repository/
│   │   │   │   └── usecase/
│   │   │   └── data/
│   │   │       ├── adb/
│   │   │       ├── discovery/
│   │   │       ├── local/
│   │   │       ├── repository/
│   │   │       ├── security/
│   │   │       └── util/
│   │   └── AndroidManifest.xml
│   └── build.gradle.kts
├── gradle/
├── build.gradle.kts
├── settings.gradle.kts
└── README.md
```

9.1 Key classes and responsibilities

Class / interface	Responsibility
TvDevice	Stable profile: id, friendly name, manufacturer/model if known, last known endpoint, connection status.
TvEndpoint	Current IP address and port for the selected ADB connection endpoint; not the pairing endpoint.
PairingRequest	Temporary pairing address, pairing port and one-time code; do not persist the code.
TvControlClient	Abstraction for pair, connect, send command, send text, screenshot and disconnect.
AdbTvControlClient	Concrete adapter around the selected ADB library.
TvDiscoveryRepository	NSD/mDNS discovery and manual endpoint validation.
TvRepository	Coordinates device profiles and active TV state.
RemoteViewModel	Exposes UI state and dispatches remote actions.
SecureCredentialStore	Stores ADB host identity securely; supports deletion and rotation.
DeviceStore	Persists non-secret TV metadata with DataStore or Room.

9.2 State model

```
sealed interface ConnectionState {
    data object Disconnected : ConnectionState
    data object Discovering : ConnectionState
    data object Pairing : ConnectionState
    data object Connecting : ConnectionState
    data class Connected(val deviceId: String) : ConnectionState
```

```
        data class Failed(val reason: UserFacingError) : ConnectionState
    }
```

10. Build the project from the ground up

10.1 Install the free development environment

1. Download Android Studio from the official Android Developers website and install the stable release.
2. Open SDK Manager and install the Android SDK Platform for the selected compileSdk, Android SDK Build-Tools and Android SDK Platform-Tools.
3. Install a supported JDK through Android Studio's Gradle JDK configuration; use the version recommended by the selected Android Gradle Plugin.
4. Create a new project using Empty Activity with Jetpack Compose enabled and Kotlin selected.
5. Choose an application ID such as `com.example.nexusremote`; select minSdk only after checking the ADB library's Android runtime requirements.
6. Run the empty app on a physical Android phone or emulator before adding networking code.
7. Initialize Git and commit the clean starter project.

10.2 Gradle dependency strategy

Use the Compose BOM and version catalog where possible. Pin compatible stable versions at implementation time; avoid copying arbitrary version numbers from old tutorials. The following is a dependency-category plan rather than a promise that every coordinate is a current release:

Category	Dependency / library family	Status
Compose UI	androidx.compose BOM, ui, foundation, material3, ui-tooling	Official AndroidX; free.
Lifecycle	androidx.lifecycle:lifecycle-viewmodel-compose	Official AndroidX; free.
Navigation	androidx.navigation:navigation-compose	Official AndroidX; free.
Coroutines	kotlinx-coroutines-android	Kotlin ecosystem; free.
DataStore	androidx.datastore:datastore-preferences	Official AndroidX; free.
Room (optional)	androidx.room:room-runtime, room-ktx, compiler	Official AndroidX; free.
ADB client	adb-kt or selected compatible ADB host library	Third-party; validate license and maintenance.
Testing	junit, androidx.compose.ui:ui-test-junit4, coroutines-test	Free testing tools.

10.3 Suggested implementation order

Sprint	Deliverable	Exit criteria
0 – Feasibility spike	Connect from a minimal Android test app to one real TV using the selected ADB library.	Modern pairing + TLS connect + one shell command demonstrated; library licence reviewed.
1 – App shell	Compose theme, navigation, Home, Discover and Settings screens.	Build succeeds; previews and navigation tests pass.
2 – Pairing	Pairing form, validation, pairing state, credential persistence.	Successful pair, invalid code and revoked authorization cases handled.
3 – Remote MVP	D-pad, OK, Back, Home, volume and media controls.	Commands work reliably on a real TV; no main-thread blocking.
4 – Device profiles	Save, rename, reconnect, remove TV.	Profiles survive app restart; credentials removed on forget.
5 – Advanced controls	App launcher, text entry, screenshot.	Each feature validated per device; unsupported behaviour reported.
6 – Hardening	Discovery, permission UX, retries, accessibility, tests and release build.	Test matrix passes; privacy/security review complete.

11. Testing and quality strategy

Test on physical televisions. Emulators are useful for UI and domain tests but cannot prove ADB pairing, vendor firmware behaviour or real Wi-Fi discovery.

11.1 Device test matrix

Test class	Suggested coverage	What to record
Android TV / Google TV brands	At least 2–3 real devices across different vendors, if available.	Brand, model, Android version, firmware, Wireless Debugging menu path.
Pairing	Correct code, wrong code, expired code, pairing cancelled, authorization revoked.	Pairing result, error mapping, recovery path.
Network	2.4 GHz, 5 GHz, router restart, phone roaming, guest Wi-Fi, AP isolation.	Discovery latency, connection success, reconnect behaviour.
Remote controls	D-pad, select, home, back, volume, mute, media, long press.	Command latency, missed repeats, unsupported actions.
App launch	Different app packages, absent app, app update, launcher restrictions.	Launch success and user-facing error.
Text input	Letters, spaces, punctuation, emoji, non-Latin scripts, long text.	Supported character coverage and escaping correctness.
Screenshot	Home screen, settings, streaming app, protected playback.	Image transfer, memory use, DRM limitations.
Lifecycle	App background/foreground, screen lock, process death, TV sleep/reboot.	Session cleanup and reconnect.

11.2 Performance and reliability targets (proposed)

- Remote button command round-trip: target median under 250 ms on a healthy home LAN; measure rather than promise.
- Repeated button presses: no unbounded command queue; cancel stale repeats when user lifts finger.
- Reconnect: bounded retry with exponential backoff and clear timeout state.
- Memory: screenshots decoded with sampling or size limits; do not retain multiple full-resolution captures unnecessarily.
- Stability: no crash on malformed IP, closed port, TV sleep, pairing timeout or network change.
- Accessibility: TalkBack labels, sufficient contrast, scalable text and keyboard navigation.

11.3 Minimum release acceptance checklist

- Pairing and connection are tested on at least one real Android TV and one Google TV model where available.
- All P0 remote actions are validated and unsupported actions are clearly surfaced.
- Private key and pairing data do not appear in logs or UI diagnostics.
- App handles network permission denial and re-request correctly for the target Android version.
- Forget device deletes the associated stored host identity or trust record as designed.
- README documents TV setup, limitations, supported devices and troubleshooting.
- Release APK is signed and tested on a clean install and upgrade path.

12. Risks, constraints and mitigations

Risk / constraint	Impact	Mitigation
ADB client library not suitable for Android app runtime	Direct phone-to-TV control cannot be shipped as planned.	Make library validation Sprint 0; keep ADB adapter interface replaceable; retain gateway fallback.
TV OEM disables or restricts Wireless Debugging	Some target devices cannot pair.	Publish a tested compatibility list; manual legacy path only where safe and explicitly enabled.
Dynamic ADB ports / discovery blocked	Hardcoded endpoint fails after reboot or network changes.	Use mDNS/NSD where possible; manual pairing and reconnect UI; never hardcode 5555.
ADB is developer interface, not consumer remote API	Commands may change or be restricted by firmware.	Keep a capability matrix; degrade gracefully; consider Android TV Remote Protocol as future alternative.
Power-on while TV is asleep	ADB connection may not be available.	Document limitation; investigate HDMI-CEC, Wake-on-LAN or IR only as optional device-specific fallback.
Text input escaping / Unicode	Incorrect text or command injection risk.	Never concatenate raw input; use vetted argument encoding and test character sets.
DRM / protected display capture	Screenshots may be blank or incomplete.	Mark screenshots experimental; do not claim protected playback capture.
Local-network permission changes	LAN access may be blocked on newer Android versions.	Review targetSdk permission guidance before each release; implement permission rationale and denial UX.
TV firmware variation	One command may behave differently across models.	Real-device matrix, capability flags and user-facing unsupported status.

12.1 Future compatibility path

Although the first version is Android/Google TV only, keep the domain interface protocol-neutral. A later release may add a separate Android TV Remote Protocol adapter if it is more appropriate for consumer-grade remote functions. Do not mix that protocol with ADB internals. Samsung Tizen, LG webOS and Roku require separate platform integrations and are explicitly outside this Android-only implementation scope.

13. Delivery roadmap and milestones

Milestone	Focus	Output
M0 – Feasibility	ADB library, pairing, TLS, shell command on one physical TV	Technical decision record + proof-of-concept APK
M1 – Foundation	Android Studio project, Compose theme, navigation, architecture	Runnable app shell
M2 – Connectivity	Manual pairing form, connection manager, secure credentials	Pair/reconnect flow
M3 – Remote MVP	D-pad, OK, Home, Back, volume and media controls	Usable first remote
M4 – Productivity features	App launcher, text input, screenshot proof-of-concept	Advanced control set
M5 – Discovery and polish	NSD/mDNS, device list, error states, accessibility	Near-complete app
M6 – Validation	Multi-device tests, security review, docs, signed build	Release candidate

13.1 Suggested team / individual workstreams

- Android UI: Compose screens, theme, navigation, accessibility and responsive layouts.
- Connectivity: ADB library evaluation, pairing flow, transport session and command execution.
- Data/security: host key storage, TV profiles, DataStore/Room and forget/revoke flows.
- QA: physical TV matrix, network scenarios, regression tests and issue tracking.
- Documentation: setup guide, supported-device matrix, privacy statement and release notes.

13.2 Definition of MVP complete

The MVP is complete when a user can install the app, pair with a validated Android/Google TV over Wireless Debugging, reconnect on the same trusted network, and use the core remote controls reliably. App launcher, keyboard and screenshot are valuable but should not delay proof of the core ADB transport.

14. Generated project concept board

The following generated visual is a high-level concept board for the NEXUS project. It combines the product vision, architecture, stack, feature map, screen concepts and roadmap. It is intended to guide planning and discussion; the written specification in this document is authoritative where a generated graphic contains simplified or illustrative details.



Concept board generated for this project brief. Confirm all technical details against the implementation notes and official references in Section 16.

15. Developer workflow and repository practices

15.1 Local development loop

1. Create a feature branch for each vertical slice (for example, `feature/adb-pairing` or `feature/remote-dpad`).
2. Implement domain interfaces and fake/test implementations before wiring real ADB operations into the UI.
3. Use Android Studio previews for Compose layout iteration and run UI tests on emulator.
4. Run integration tests against a physical phone and TV for all network and ADB changes.
5. Record TV model, OS version, router configuration and reproduction steps for every device-specific defect.
6. Keep dependency versions pinned and review library updates before upgrading.
7. Create a signed release candidate only after the security checklist and compatibility notes are updated.

15.2 Logging strategy

Log	Allowed	Never log
Connection	State transitions, elapsed time, safe endpoint metadata if user opts into diagnostics.	Pairing code, private key, authentication tokens.
Commands	Command enum name and result code.	Arbitrary shell content or raw user text.
Errors	Sanitized exception type and stable error code.	Sensitive TV content or credential-bearing buffers.
Diagnostics export	Device model/OS only with user consent.	Wi-Fi password, unrelated LAN device inventory.

15.3 Documentation deliverables

- README: project purpose, build/run steps, supported TV setup and known limitations.
- Architecture Decision Record: chosen ADB library, pairing implementation, key storage and fallback decision.
- Device Compatibility Matrix: tested model, firmware, connection mode and feature results.
- Threat Model: assets, trust boundary, local network assumptions, abuse cases and mitigations.
- Release Notes: supported features, breaking changes and known issues.

16. Technical references and validation sources

The following primary and project sources were consulted for this blueprint. They should be rechecked during implementation because Android platform behaviour, target SDK requirements and third-party library maintenance can change.

Ref	Source	Use in this document
[1]	Android Developers – Android Debug Bridge (ADB) https://developer.android.com/tools/adb	ADB host/client/server model, shell services and screenshot capture.
[2]	Android Developers – Run apps on a hardware device / Wi-Fi debugging https://developer.android.com/studio/run/device	Android 11+ wireless pairing workflow and developer workstation pairing.
[3]	AOSP – Architecture of ADB Wi-Fi AOSP adb_wifi.md	Pairing protocol, TLS transport, service advertisements and separate endpoints.
[4]	Android Developers – Jetpack Compose setup https://developer.android.com/develop/ui/compose/setup	Compose project setup and Android Studio workflow.
[5]	Kotlin – Kotlin for Android https://kotlinlang.org/docs/android-overview.html	Kotlin and Jetpack ecosystem context.
[6]	Android Developers – Local network permission https://developer.android.com/privacy-and-security/local-network-permission	Local-network access and Android 17/API 37 permission guidance.
[7]	Android Developers – Wi-Fi permissions Wi-Fi permissions guide	Nearby Wi-Fi permission considerations.
[8]	Google AOSP – adblib source tree AOSP adblib	Official ADB host library context; not assumed to be a drop-in Android app SDK.
[9]	adb-kt – third-party Kotlin ADB client documentation https://github.com/rhythmcache/adb-kt	Candidate implementation to evaluate for on-device ADB host integration.

Reference interpretation notes

- Official Android documentation describes ADB and wireless debugging primarily as a developer workflow between a workstation host and an Android device. NEXUS reverses the host role by implementing an ADB host inside an Android phone app; this is a separate engineering requirement.
- ADB shell commands and key events are illustrative. Actual command availability and behaviour must be tested on target TV firmware.
- The `adb-kt` repository is a candidate, not an endorsement. Review its current code, releases, issue tracker, licence and security posture before integration.
- Generated UI images are visual concepts, not evidence that a working app, API or connection has already been implemented.

17. Final recommendation

Build NEXUS as a native Kotlin Android application using Jetpack Compose and Material 3. Keep the product local-first and avoid a paid cloud backend. The highest-risk work is not the remote UI; it is implementing a reliable, secure ADB host inside the phone app and validating it against real Android/Google TV firmware.

The project should begin with a small proof-of-concept that performs Wireless Debugging pairing, establishes the authenticated connection and sends one safe key event. Only after that succeeds should the team invest in the complete Compose experience.

> **Recommended first engineering task**
> Create a minimal Android Studio Kotlin app and evaluate the selected ADB client library on a real Android/Google TV. Demonstrate: pair with code → establish TLS ADB connection → send Home key event → disconnect cleanly. Document library version, licence, device model, Android version and limitations.

If the direct ADB host approach is successful, proceed with the native-only architecture. If it is not, keep the same Kotlin UI and domain layer but add an optional local gateway adapter as a fallback. This protects the product plan without hiding the main technical risk.

NEXUS TV REMOTE

A native, local-network remote for Android TV and Google TV — designed around secure pairing, extensible control commands and a polished Android experience.

End of document • Version 1.0